

DATABRICKS GROUP PROJECT

OULAD Education Analytics Lakehouse Project

Databricks Medallion Architecture

Bronze → Silver → Gold • Delta Lake • PySpark • SQL Analytics

Domain	Education / EdTech
Dataset Format	CSV (7 Tables)
Architecture	Medallion (Bronze / Silver / Gold)
Team Size	3 Members
Technology	Databricks, PySpark, Delta Lake, SQL
Source	Open University, UK — analyse.kmi.open.ac.uk/open_dataset

BRONZE

Raw Ingestion
7 Delta Tables
10M+ rows ingested

SILVER

Clean & Transform
7 Cleaned Tables
MERGE + Schema Evolution

GOLD

Analytics & KPIs
8 KPI Tables
Window Functions + Time Travel

Open University Learning Analytics Dataset (OULAD)

Student Performance & Engagement Pipeline • 7 Tables • 10M+ Activity Records • 173K+
Assessment Records

Covers: Dropout Detection • Engagement Analytics • At-Risk Flagging • Course KPI Reporting

1. Business Pitch

Universities and online learning platforms struggle to consolidate data scattered across student registration systems, assessment platforms, learning management systems, and engagement logs. Administrators cannot easily track course completion, identify at-risk students, or correlate engagement with academic outcomes.

This project builds a Databricks Lakehouse on top of the Open University Learning Analytics Dataset (OULAD) to create a single trusted source of truth. By integrating student demographics, course structure, assessment records, and Virtual Learning Environment (VLE) activity logs, the pipeline enables data-driven decisions about student support, curriculum design, and dropout prevention.

2. Dataset Summary

Source: Open University Learning Analytics Dataset (OULAD) — Open University, UK

Link: https://analyse.kmi.open.ac.uk/open_dataset

Table / File	Row Count	Key Columns	Data Type
courses.csv	~22	code_module, code_presentation	Master/Dimension
studentInfo.csv	~32,500	id_student, code_module, final_result	Dimension
assessments.csv	~200	id_assessment, code_module, weight	Dimension
studentAssessment.csv	~173,000	id_student, id_assessment, score	Fact
studentRegistration.csv	~32,500	id_student, date_registration, date_unregistration	Fact
vle.csv	~6,300	id_site, activity_type, week_from, week_to	Dimension
studentVle.csv	10,000,000+	id_student, id_site, date, sum_click	Fact (Large)

3. Team Roles & Responsibilities

Member	Role	Primary Layer	Key Deliverable
M1	Ingestion Engineer	Bronze	Load all 7 CSVs to Delta tables with audit columns, register in Metastore
M2	Transformation Engineer	Silver	Clean, join, and apply MERGE + Schema Evolution across all 7 Silver tables
M3	Analytics Engineer	Gold	Build Gold KPI tables, Window Functions, Time Travel, README, PPT

4. Technical Requirements

Requirement	M1 — Bronze	M2 — Silver	M3 — Gold
PySpark DataFrame API	Load 7 CSVs	Silver transforms	Gold aggregations
Delta Tables	7 Bronze tables	7 Silver tables	7+ Gold KPI tables

INSERT / UPDATE / DELETE	—	DML on students/scores	—
MERGE INTO (Incremental)	—	Primary owner	—
Schema Evolution	—	Add derived columns	—
Time Travel	—	—	Primary owner
SQL Analytics (CTEs)	Validation queries	Quality checks	All Gold KPI queries
Window Functions	—	—	RANK, DENSE_RANK, LAG, LEAD
Joins & Aggregations	—	7-table Silver join	Gold aggregations
Metastore Registration	7 Bronze tables	7 Silver tables	7+ Gold tables

5. Execution Steps

Step 1 — Load Raw Data into Bronze

- Download all 7 OULAD CSV files from the official dataset link
- Upload files to DBFS under /FileStore/oulad/raw/
- Read each file using `spark.read.option('header', True).csv(path)` with `inferSchema=True`
- Add audit columns: `ingestion_timestamp`, `source_file`, `layer='bronze'`
- Save as Delta tables to `/delta/bronze/` and register all 7 in Metastore
- Validate row counts: source CSV vs Delta table using `SELECT COUNT(*)`

Step 2 — Clean & Transform Silver Layer

- Remove NULLs in primary keys: `id_student`, `id_assessment`, `code_module`
- Handle NULL scores: set to 0 or flag with `is_submitted` column
- Standardise categorical values: `imd_band`, `age_band`, `final_result`
- Trim all string fields; remove duplicate records
- Deduplicate studentVle by aggregating `sum_click` per (`id_student`, `id_site`, `date`)
- Add derived columns: `enrollment_duration`, `risk_flag`, `weighted_score`

Step 3 — Join Tables in Silver

- Join `silver_students` + `silver_registrations` + `silver_student_scores` + `silver_assessments` + `silver_engagement` on `id_student`, `code_module`, `id_assessment`
- Confirm no fan-out row multiplication after joins
- Validate all join keys are non-null in Silver tables

Step 4 — MERGE for Incremental Loads

- Use `MERGE INTO silver_student_scores` for new daily assessment batch submissions
- Simulate a small batch (~50 new student score records) and run `MERGE`
- Verify only new/changed rows are inserted or updated

Step 5 — Delta Lake Operations

- Perform `INSERT`, `UPDATE`, `DELETE` on Silver Delta tables (e.g., `UPDATE final_result` after re-grading)
- Apply Schema Evolution: add `risk_flag` and `enrollment_duration` columns safely using `mergeSchema`
- Run `DESCRIBE HISTORY` on `silver_students` Delta table
- Demo Time Travel: `VERSION AS OF` and `TIMESTAMP AS OF` on `bronze_student_info`

Step 6 — Build Gold KPI Layer

- Write all KPI queries using SQL CTEs and PySpark aggregations
- Save all Gold tables to `/delta/gold/` and register in Metastore
- Apply Window Functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD` on student scores

6. Gold Layer KPI Outputs

#	KPI Name	Description	Business Value
1	Pass Rate by Module	% students with final_result = 'Pass' or 'Distinction' per module, ranked with DENSE_RANK	Identifies underperforming courses
2	Dropout Rate Analysis	% students with date_unregistration NOT NULL; AVG enrollment duration by region	Early intervention trigger
3	Avg Assessment Score	Mean score per module per assessment type (TMA, CMA, Exam)	Curriculum difficulty analysis
4	Engagement Score	Total and weekly clicks per student across all VLE resources using LAG/LEAD trends	Predicts student success
5	At-Risk Student Flag	Students with total_clicks < threshold AND avg_score < 40; ranked with ROW_NUMBER	Student support prioritisation
6	Submission Rate	% of assessments submitted on time per student per module	Identifies habitual non-submitters
7	Top VLE Content Types	VLE resource types with highest total clicks; ranked with RANK()	Content strategy optimisation
8	Executive KPI Summary	Total students, pass rate, avg engagement, dropout rate, top module by completion	Executive reporting dashboard

7. Why This Dataset Is a Strong Choice

- Real-world academic behavioural data from the Open University, UK — covering actual student interactions and outcomes
- Large enough for meaningful analytics: 173,000+ assessment records and 10 million+ VLE click logs
- Rich join relationships across 7 tables using id_student, code_module, id_assessment, and id_site
- Supports complex queries: dropout detection, engagement trends, assessment difficulty, student performance ranking
- Contains intentional data quality issues — NULLs, duplicates, inconsistent categories — ideal for a realistic Silver cleaning pipeline
- Strong business storytelling value: directly applicable to EdTech, HE institutions, and student success platforms
- Covers all required Databricks features: Delta, MERGE, Time Travel, Schema Evolution, Window Functions

8. Deliverables

Deliverable	Details	Owner
Bronze Notebook	7 Delta tables loaded, row counts validated, audit columns added, Metastore registered	M1
Silver Notebook	7 cleaned Delta tables, MERGE INTO implemented, Schema Evolution applied, join validation complete	M2
Gold Notebook	8 KPI tables, Window Functions, Time Travel demo, all tables registered in Metastore	M3
SQL Scripts	All KPI queries using CTEs, DML statements (UPDATE/DELETE), MERGE logic, Time Travel queries	M3
Screenshots	Row count validation, before/after cleaning, Time Travel output, MERGE metrics, Gold KPI results	All
README	Project overview, dataset description, setup steps, pipeline flow diagram, KPI definitions	M3
Presentation (PPT)	5–7 slides: overview, architecture diagram, Bronze/Silver/Gold highlights, KPIs, live demo	All
GitHub Repository	Clean repo with all notebooks, SQL scripts, and README pushed; commit history showing contributions	All